VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT REPORT
INTELLIGENT SYSTEM (CO5134)

HEART DISEASE AND STROKE DIAGNOSIS WITH RANDOM FOREST

Instructor: Prof. Quan Thanh Tho
Students: Vuong Minh Toan 2491057
Tran Nguyen Huan 2370692

HO CHI MINH, November 2024

Contents

1 Stakeholders identification and potential benefits for them **3**
1.1 Stakeholder . . . 3
1.2 Requirement Analysis . . . 3
1.2.1 Functional Requirements . . . 3
1.2.2 Non-Functional Requirements . . . 4

2 System design **7**
2.1 Frontend: React.js . . . 7
2.2 Backend: Flask . . . 7
2.3 System Workflow . . . 9
2.4 Deployment and Maintenance . . . 10
2.5 Source Code and Demo . . . 10

3 Data description and preprocessing **11**
3.1 Dataset Description . . . 11
3.2 Data Preprocessing Steps . . . 12

4 Details of AI pipeline for training **21**
4.1 Model detail . . . 21
4.2 Front end . . . 24
4.2.1 Overall . . . 24
4.2.2 Application illustration . . . 25
4.3 Back end . . . 27
4.3.1 Overall . . . 27
4.3.2 Backend processing workflow . . . 27

5 Evaluation results **28**
5.1 Heart Disease Diagnosis . . . 29
5.2 Stroke Disease Diagnosis . . . 31

List of Figures

2.3.1 Overall flow 9
2.3.2 Heart disease prediction flow 9
3.2.1 The statistics of the number of features with NaN values (heart.csv) . . . 13
3.2.2 The statistics of the number of features with NaN values (stroke.csv) . . . 14
3.2.3 heatmap (heart.csv) 15
3.2.4 heatmap (stroke.csv) 15
3.2.5 Heart Disease by Gender 16
3.2.6 Heart Disease Distribution 17
3.2.7 Stroke by Gender 18
3.2.8 Stroke Distribution in dataset 19
3.2.9 The result after performing sampling using SMOTE 19
3.2.10 Stroke Distribution in dataset after performing sampling using SMOTE . 20
4.1.1 LazyPredict results for quick model comparison, testing, and selection on the heart dataset 22
4.1.2 LazyPredict results for quick model comparison, testing, and selection on the stroke dataset 23
4.1.3 Enter Caption 24
4.1.4 Enter Caption 24
4.2.1 Homepage Interface 25
4.2.2 Heart Diagnosis Interface 26
4.2.3 Heart Diagnosis Prediction Result 26
5.1.1 Confusion Matrix of Heart Disease Model 29
5.1.2 The Accuracy of Heart Disease Model 29
5.1.3 ROC of Heart Disease Model 30
5.2.1 Confusion Matrix of Stroke Disease Model 31
5.2.2 The Accuracy of Stroke Disease Model 31
5.2.3 ROC of Stroke Disease Model 32

Section 1

Stakeholders identification and potential benefits for them

1.1. Stakeholder

- **General Public (Community Users):**
  - *Role:* Use the application to check for potential risks of heart disease based on their health data.
  - *Benefits:*
    - Easy access to health risk assessments without needing specialized medical knowledge.
    - Awareness of potential health issues, prompting timely medical consultation.
    - Access to basic health insights with minimal financial cost.
- **Health Awareness Campaigns (NGOs, Community Organizations):**
  - *Role:* Use the application for community health initiatives.
  - *Benefits:*
    - Leverage data for identifying at-risk individuals in underserved communities.
    - Support educational efforts around preventive healthcare.
- **Local Governments and Public Health Authorities:**
  - *Role:* Monitor community health trends using anonymized data.
  - *Benefits:*
    - Data-driven insights to design better public health interventions.
    - Early detection of broader health risks at the community level.

1.2. Requirement Analysis

1.2.1. Functional Requirements

- Input personal health data (e.g., age, gender, blood pressure, cholesterol levels).
- Display prediction results with health insights.
- Generate downloadable and printable health reports.

1.2.2. Non-Functional Requirements

- Intuitive user interface with clear navigation.
- Fast response time for predictions (e.g., under 5 seconds).
- 99.9% uptime with minimal maintenance downtime.

Random Forest Algorithm for Heart Disease and Stroke Prediction

Overview of Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy and robustness. Each tree is trained on a randomly selected subset of data and features, reducing overfitting and enhancing generalization.

Algorithm Steps

Training Phase

1. **Input:**
   - Dataset $D = \{(X_1, Y_1), (X_2, Y_2), \ldots, (X_n, Y_n)\}$, where X_i represents the features (e.g., blood pressure, cholesterol levels, etc.) and Y_i represents the labels (presence or absence of heart disease).
   - Parameters:
     - Number of trees T,
     - Number of samples N per tree,
     - Maximum tree depth $d_{\max}$,
     - Number of randomly selected features m at each split.
2. **Bootstrap Sampling:**
   - Randomly sample N data points with replacement from D to create a new training subset D_t for each tree t.
3. **Feature Selection:**
   - At each node of the decision tree, randomly select m features out of the total M features for the splitting criterion.
4. **Tree Construction:**
   - Build a decision tree h_t using D_t and the selected features.
   - Split nodes to minimize a metric such as Gini impurity (for classification) or mean squared error (for regression) until a stopping condition is met (e.g., maximum depth $d_{\max}$).
5. **Repeat:**

- Repeat steps 2–4 for T trees to form the forest $F = \{h_1, h_2, \ldots, h_T\}$.

Prediction Phase

1. For a new sample x:
   - Pass x through all trees h_t in the forest.
   - Aggregate predictions:
     - For classification: Use majority voting.
     - For regression: Compute the average of predictions.
2. Output the final prediction.

Benefits for Heart Disease Prediction

1. **Handles Complex Relationships:** Models nonlinear interactions between features (e.g., age, cholesterol levels) and heart disease presence.
2. **Feature Importance:** Ranks critical predictors of heart disease, such as hypertension or smoking status.
3. **Robustness to Noise:** Less sensitive to noisy data and outliers due to its ensemble nature.
4. **Avoids Overfitting:** Reduces overfitting by averaging multiple trees trained on different subsets of data.
5. **Works with High-Dimensional Data:** Handles datasets with many features, making it suitable for analyzing diverse patient profiles.
6. **Handles Missing Data:** Maintains accuracy even with missing data by leveraging decision trees.
7. **Versatility:** Works for both classification (predicting heart disease presence or stroke potential) and regression (predicting risk scores).
8. **Scalability:** Parallelizable training allows it to handle large datasets effectively.

Application Example

Random Forest can use features such as:

- **Demographic Data:** Age, gender.
- **Clinical Data:** Blood pressure, cholesterol levels, fasting blood sugar.
- **Lifestyle Data:** Smoking status, alcohol consumption.
- **Test Results:** ECG readings, heart rate, echocardiogram results.

Section 2

System design

The system for the community website integrates a React.js frontend and a Flask backend to deliver a user-friendly, efficient, and functional heart disease prediction application. Below is a description of the components and how the system works:

2.1. Frontend: React.js

The frontend handles user interaction and provides an engaging interface. The main components include:

- **Interactive Forms**: Users can input personal health metrics and upload diagnostic images.
- **Dynamic Pages**: Includes sections like the home page, prediction tool, results display, educational resources, and feedback forms.
- **Responsive Design**: Ensures compatibility across devices (mobile and desktop).
- **API Integration**: Communicates seamlessly with the backend for data exchange using Axios.

2.2. Backend: Flask

The backend powers the core logic and integrates the machine learning model to predict heart disease risks. Key features include:

- **API Endpoints**: Provides endpoints for prediction, resources, feedback, and server health checks.
- **Machine Learning Integration**: Utilizes pre-trained models to analyze data and generate predictions.
- **Data Handling**: Ensures secure and temporary processing of sensitive data, including uploaded images.
- **Modular Architecture**: Supports scalability and future enhancements.

2.3. System Workflow

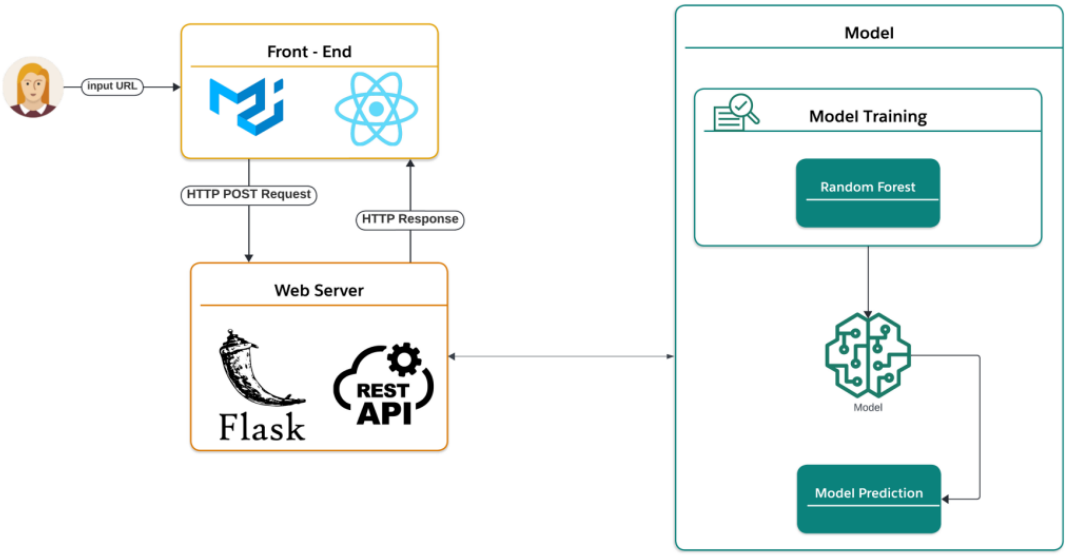

Figure 2.3.1: Overall flow

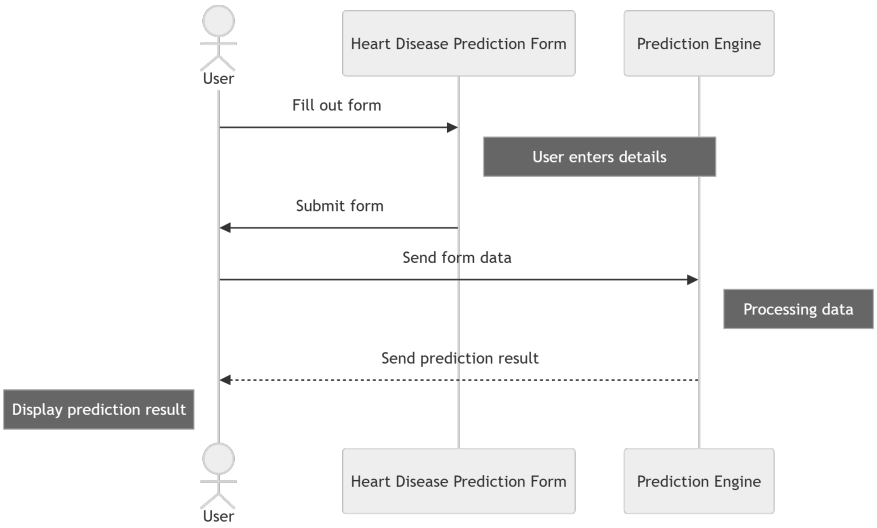

Figure 2.3.2: Heart disease prediction flow

The overall workflow of the system is as follows:

1. Users interact with the React frontend, filling out forms.
2. Data is submitted to the Flask backend via a REST API call.
3. The backend validates and preprocesses the input data.
4. The prediction model generates results based on the input data.
5. Results are returned to the frontend and displayed to the user.
6. Users can review results and download reports.

2.4. Deployment and Maintenance

The system is currently on developing environment but we can use these technologies for deployment and maintenance:

- **Frontend Deployment**: React app is built and hosted on platforms like Netlify, Vercel, or AWS S3.
- **Backend Deployment**: Flask app is deployed using Gunicorn or uWSGI, hosted on AWS EC2, Heroku, or Azure.
- **Monitoring**: Tools like Sentry (error tracking), Prometheus, and Grafana (backend monitoring) are used.
- **Analytics**: Frontend user activity is tracked with Google Analytics.

2.5. Source Code and Demo

- Link to source code: Google Drive
- Link to demo: Youtube

Section 3

Data description and preprocessing

3.1. Dataset Description

The heart.csv and stroke.csv dataset commonly found on Kaggle is typically used for predicting the presence of diagnosed disease in patients based on various clinical and physiological features. Each row represents a patient, and the columns represent different attributes or features that may indicate health.

Features in heart.csv (1025 samples):

Feature	**Description**
age	Age of the patient (years)
sex	Gender of the patient (1 = Male, 0 = Female)
cp	Chest pain type (0-3: different types of chest pain)
trestbps	Resting blood pressure (mm Hg)
chol	Serum cholesterol (mg/dL)
fbs	Fasting blood sugar ¿ 120 mg/dL (1 = True, 0 = False)
restecg	Resting electrocardiographic results (0-2)
thalach	Maximum heart rate achieved
exang	Exercise-induced angina (1 = Yes, 0 = No)
oldpeak	ST depression induced by exercise relative to rest
slope	Slope of the peak exercise ST segment (0-2)
ca	Number of major vessels (0-3) colored by fluoroscopy
thal	Thalassemia status (0 = Normal, 1 = Fixed defect, 2 = Reversible defect)
target	Presence of heart disease (1 = Disease, 0 = No disease)

Table 3.1.1: Feature descriptions for heart disease dataset

Features in stroke.csv (5110 samples):

Feature	Description
gender	Gender of the patient (e.g., 'Male', 'Female')
age	The age of the patient (in years)
hypertension	Whether the patient has hypertension (1 for 'Yes', 0 for 'No')
heart_disease	Whether the patient has heart disease (1 for 'Yes', 0 for 'No')
ever_married	Whether the patient has ever been married (e.g., 'Yes', 'No')
work_type	The type of work the patient does (e.g., 'Private', 'Self-employed', 'Govt_job', 'children', 'Never_worked')
residence_type	Whether the patient lives in an urban or rural area (e.g., 'Urban', 'Rural')
avg_glucose_level	The average glucose level of the patient
bmi	The Body Mass Index of the patient
smoking_status	The smoking status of the patient (e.g., 'formerly smoked', 'never smoked', 'smokes', 'unknown')
stroke	The target variable (1 if the patient had a stroke, 0 if not)

Table 3.1.2: Feature descriptions for stroke prediction dataset

3.2. Data Preprocessing Steps

We will train this model using Visual Studio Code, leveraging libraries such as Numpy, Seaborn, Pandas, scikit-learn, and more.

Step 1: Loading the Dataset

- Use Pandas to load the heart.csv file and stroke.csv

```
df = pd.read_csv('file_name.csv')
```

Step 2: Mapping

- Mapping the values of the 'target' column for heart.csv

```
df['target'] = df['target'].map({0: 0, 1: 1, 2: 1, 3: 1, 4: 1})
```

- Mapping the values of the 'ever_married' column for stroke.csv

```
df['ever_married'] = df['ever_married'].map( {'Yes':1,'No':0})
df['ever_married'].value_counts()
```

Because some features in stroke.csv have many options, mapping will not be effective in building the model, so we will use **One-Hot Encoding** to divide into many sub-features to train the model more effectively.

```
df = pd.get_dummies(df).astype(int)
```

Feature	Meaning
gender_Female	1 if the individual is female, 0 otherwise
gender_Male	1 if the individual is male, 0 otherwise
work_type_Govt_job	1 if the individual works in a government job, 0 otherwise
work_type_Never_worked	1 if the individual has never worked, 0 otherwise
work_type_Private	1 if the individual works in the private sector, 0 otherwise
work_type_Self-employed	1 if the individual is self-employed, 0 otherwise
work_type_children	1 if the individual is a child (not working), 0 otherwise
residence_type_Rural	1 if the individual lives in a rural area, 0 otherwise
residence_type_Urban	1 if the individual lives in an urban area, 0 otherwise
smoking_status_Unknown	1 if the individual's smoking status is unknown, 0 otherwise
smoking_status_formerly smoked	1 if the individual smoked in the past but has quit, 0 otherwise
smoking_status_never smoked	1 if the individual has never smoked, 0 otherwise
smoking_status_smokes	1 if the individual is currently a smoker, 0 otherwise

Table 3.2.1: Feature descriptions for categorical encoding

Step 3: Handling Missing Values

Check for any missing or null values and handle them.

```
nan_counts = df.isna().sum()
print(nan_counts)
```

- **heart.csv:**

```
Số lượng giá trị NaN trong từng cột:
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
Không có giá trị NaN trong DataFrame.
```

Figure 3.2.1: The statistics of the number of features with NaN values (heart.csv)

- **stroke.csv:**

```
id                   0
gender               0
age                  0
hypertension         0
heart_disease        0
ever_married         0
work_type            0
Residence_type       0
avg_glucose_level    0
bmi                201
smoking_status       0
stroke               0
dtype: int64
```

Figure 3.2.2: The statistics of the number of features with NaN values (stroke.csv)

we have 201 NULL values in bmi column , since we have 5110 instance and bmi is numerical column we will drop NULLs data.

```
df = df.dropna()
```

Step 4: Data Visualization

- Create heatmap:

- **heart.csv:**

```
plt.figure(figsize=(20, 16))
sns.heatmap(df.drop('target', axis = 1). corr(), cmap=
    "YlGnBu", annot = True)
plt.show()
```

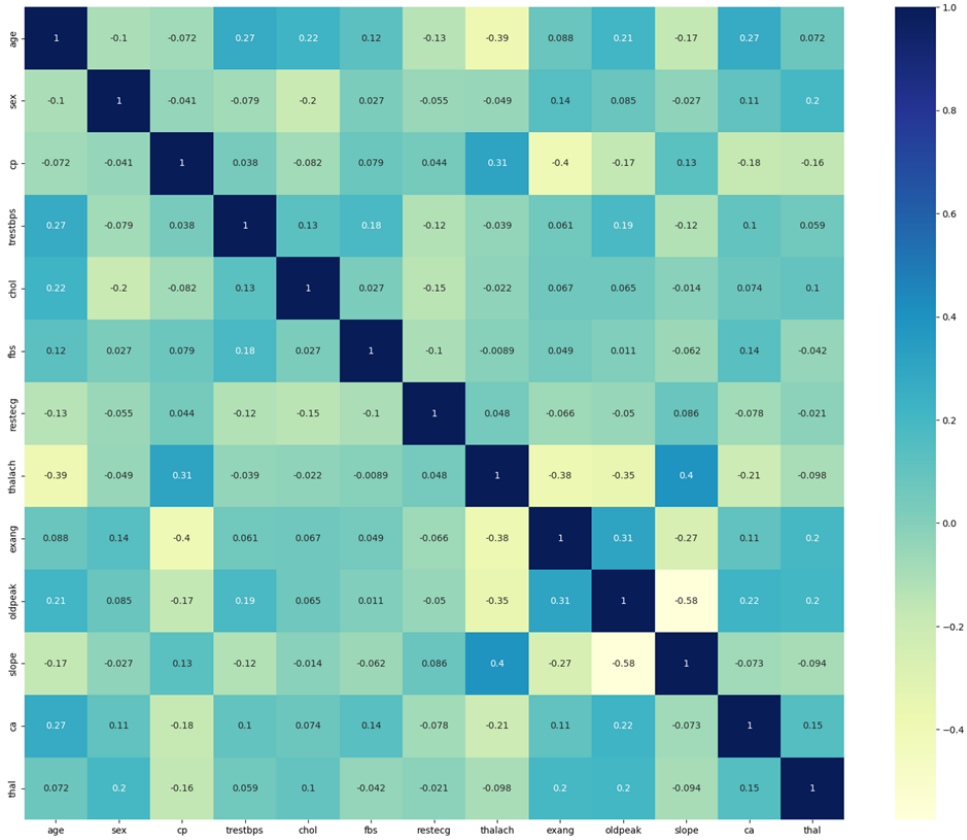

Figure 3.2.3: heatmap (heart.csv)

No variable has a correlation coefficient above 0.8, so all variables are retained.

- **stroke.csv:**

```
numeric_df = df.drop('stroke', axis=1).
    select_dtypes(include=['number'])
corr = numeric_df.corr()
plt.figure(figsize=(20, 16))
sns.heatmap(corr, cmap="YlGnBu", annot=True)
plt.show()
```

Figure 3.2.4: heatmap (stroke.csv)

No variable has a correlation coefficient above 0.8, so all variables are retained.

- Create pie chart for target value:

- **heart.csv:**

```
ColumnsForCountPlot = ['sex']
for column in ColumnsForCountPlot:
```

```
        heart_percentages = df.groupby(column)['target'].mean() * 100
        plt.figure(figsize=(8, 8))
        plt.pie(heart_percentages, labels=heart_percentages.index, autopct='%1.1f%%', startangle=140)
        plt.title(f'Heart Percentage by {column}')
        plt.show()
```

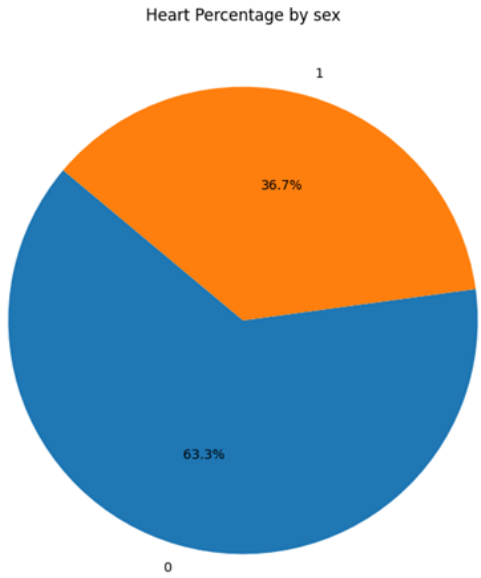

Figure 3.2.5: Heart Disease by Gender

We see that women tend to have heart disease more than men.

```
    target_counts = df['target'].value_counts()
    plt.figure(figsize=(6, 6))
    plt.pie(
        target_counts,
        labels=target_counts.index,
        autopct='%1.1f%%',
        startangle=90,
        colors=['skyblue', 'orange'],
        explode=[0.05] * len(target_counts)
    )
    plt.title('Distribution of Target Values', fontsize=14)
```

```
    plt.show()
```

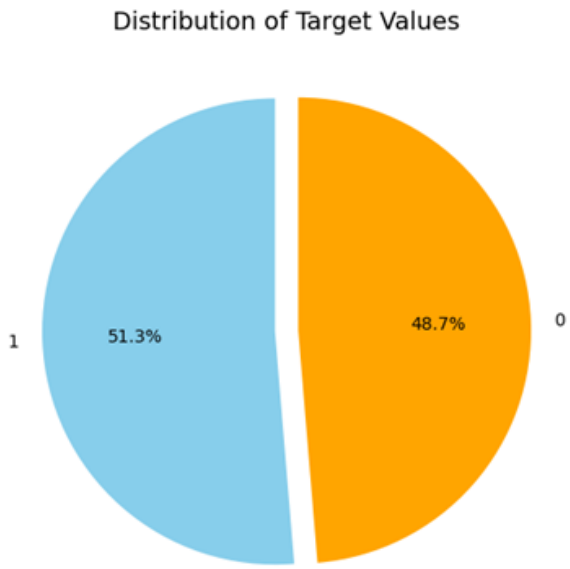

Figure 3.2.6: Heart Disease Distribution

There is no target imbalance in the dataset, so this dataset can be used to build the diagnostic model.

- **stroke.csv:**

```
    ColumnsForCountPlot = ['gender']
    for column in ColumnsForCountPlot:
        stroke_percentages = df.groupby(column)['stroke'].
           mean() * 100
        plt.figure(figsize=(8, 8))
        plt.pie(stroke_percentages, labels=
           stroke_percentages.index, autopct='%1.1f%%',
           startangle=140)
        plt.title(f'Stroke Percentage by {column}')
        plt.show()
```

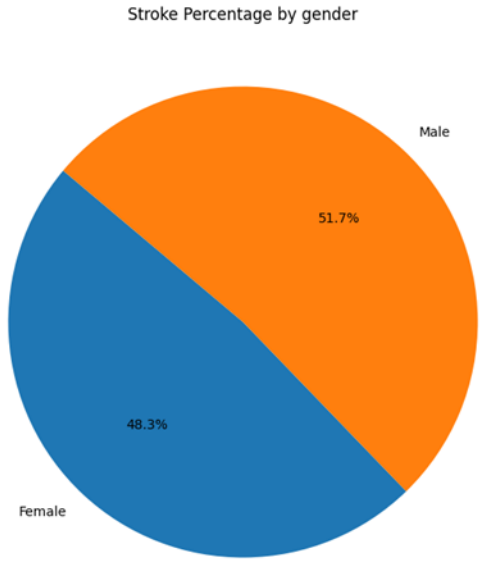

Figure 3.2.7: Stroke by Gender

We see that men and women are equally prone to stroke.

```
target_counts = df['stroke'].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(
    target_counts,
    labels=target_counts.index,
    autopct='%1.1f%%',
    startangle=90,
    colors=['skyblue', 'orange'],
    explode=[0.05] * len(target_counts)
)
plt.title('Distribution of Stroke Values', fontsize=14)
plt.show()
```

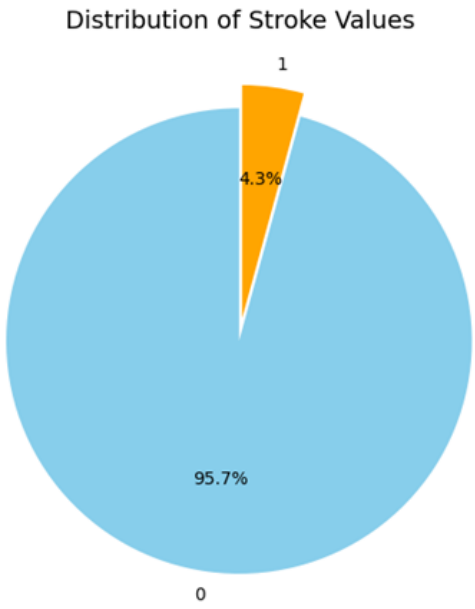

Figure 3.2.8: Stroke Distribution in dataset

Stroke Values distribution data shows a severe imbalance between the two labels. So we will use SMOTE to handle data imbalance.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(sampling_strategy="minority")
x_smote , y_smote = smote.fit_resample(x,y)
y_smote.value_counts()
```

```
stroke
1    4699
0    4699
Name: count, dtype: int64
```

Figure 3.2.9: The result after performing sampling using SMOTE

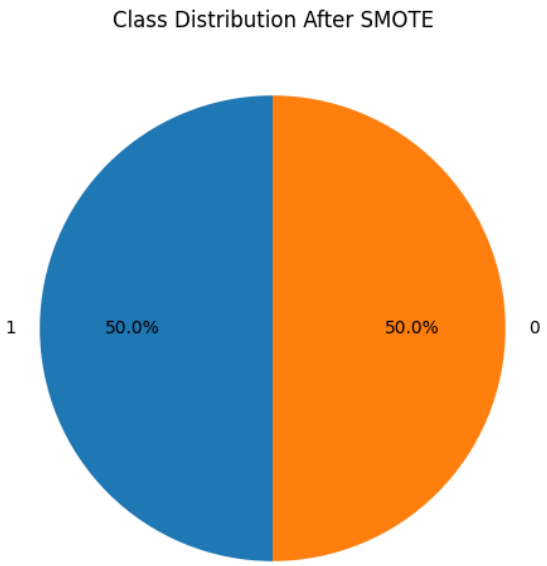

Figure 3.2.10: Stroke Distribution in dataset after performing sampling using SMOTE

Section 4

Details of AI pipeline for training

Source code front end: Click here

Source code back end: Click here

4.1. Model detail

Step 1: Splitting the Data

Split the dataset into training and testing sets.

- **heart.csv:**

```
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_state=42)
```

- **stroke.csv:**

Due to the data samples with the target value of 1 being mostly concentrated at the beginning of the dataset, we will add a shuffling step before training the model.

```
X_train , X_test , y_train , y_test = train_test_split(x_smote,y_smote,test_size=0.2,random_state=42, stratify=y_smote)
scaler=StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
X_train, y_train = shuffle(X_train, y_train, random_state=42)
X_test, y_test = shuffle(X_test, y_test, random_state=42)
```

Step 2: Use lazypredict library to quickly test and compare multiple machine learning models, choose potential machine learning models.

```
clf = LazyClassifier(verbose=0,ignore_warnings=True, custom_metric=None)
```

```
    models,predictions = clf.fit(X_train, X_test, y_train,
        y_test)
    print(models)
```

- **heart.csv:**

```
100%|██████████| 31/31 [00:00<00:00, 36.26it/s]
[LightGBM] [Info] Number of positive: 423, number of negative: 397
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.000086 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 371
[LightGBM] [Info] Number of data points in the train set: 820, number of used features: 13
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.515854 -> initscore=0.063436
[LightGBM] [Info] Start training from score 0.063436
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
                           Accuracy  Balanced Accuracy  ROC AUC  F1 Score  \
Model
ExtraTreeClassifier            1.00               1.00     1.00      1.00
DecisionTreeClassifier         0.99               0.99     0.99      0.99
BaggingClassifier              0.99               0.99     0.99      0.99
LabelPropagation               0.99               0.99     0.99      0.99
ExtraTreesClassifier           0.99               0.99     0.99      0.99
XGBClassifier                  0.99               0.99     0.99      0.99
LabelSpreading                 0.99               0.99     0.99      0.99
RandomForestClassifier         0.99               0.99     0.99      0.99
LGBMClassifier                 0.99               0.99     0.99      0.99
SVC                            0.89               0.89     0.89      0.89
...
Perceptron                       0.00
SGDClassifier                    0.02
PassiveAggressiveClassifier      0.02
DummyClassifier                  0.00
```

Figure 4.1.1: LazyPredict results for quick model comparison, testing, and selection on the heart dataset

- **stroke.csv:**

```
100%|██████████| 31/31 [00:09<00:00,  3.11it/s]
[LightGBM] [Info] Number of positive: 3759, number of negative: 3759
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.000500 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 392
[LightGBM] [Info] Number of data points in the train set: 7518, number of used features: 18
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
                             Accuracy  Balanced Accuracy  ROC AUC  F1 Score  \
Model
BaggingClassifier                0.98              0.98     0.98      0.98
ExtraTreesClassifier             0.98              0.98     0.98      0.98
RandomForestClassifier           0.98              0.98     0.98      0.98
LGBMClassifier                   0.97              0.97     0.97      0.97
KNeighborsClassifier             0.97              0.97     0.97      0.97
XGBClassifier                    0.97              0.97     0.97      0.97
SVC                              0.97              0.97     0.97      0.97
LabelPropagation                 0.96              0.96     0.96      0.96
LogisticRegression               0.96              0.96     0.96      0.96
LabelSpreading                   0.96              0.96     0.96      0.96
SGDClassifier                    0.96              0.96     0.96      0.96
CalibratedClassifierCV           0.96              0.96     0.96      0.96
LinearSVC                        0.96              0.96     0.96      0.96
QuadraticDiscriminantAnalysis    0.96              0.96     0.96      0.96
AdaBoostClassifier               0.96              0.96     0.96      0.96
ExtraTreeClassifier              0.96              0.96     0.96      0.96
...
NearestCentroid                    0.02
BernoulliNB                        0.02
GaussianNB                         0.02
DummyClassifier                    0.02
```

Figure 4.1.2: LazyPredict results for quick model comparison, testing, and selection on the stroke dataset

Most supervised machine learning models work well with this data file, so we decided to use a random forest model for our analysis.

Step 3: training model

- Create model and define optimization parameters:

```
classifier = RandomForestClassifier(random_state=42)
param_grid = {
'max_depth': [None, 10, 20, 30],
'max_features': ['sqrt', 'log2'],
'min_samples_leaf': [1, 2, 4],
'min_samples_split': [2, 5, 10],
'n_estimators': [50, 100, 200] }
```

- Set up Grid Search CV (this will help us to find the best parameter for model):

```
grid_search = GridSearchCV(estimator=classifier,
    param_grid=param_grid, cv=5, n_jobs=-1, error_score
    ='raise')
grid_search.fit(X_train, y_train)
print("Best Parameters:", grid_search.best_params_)
```

```
print("Best Cross-validation Accuracy:", grid_search.best_score_)
```

- **heart.csv:**

```
Best Parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 50}
Best Cross-validation Accuracy: 0.9817073170731707
```

Figure 4.1.3: Enter Caption

- **stroke.csv:**

```
Best Parameters: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 50}
Best Cross-validation Accuracy: 0.9749926565309097
```

Figure 4.1.4: Enter Caption

4.2. Front end

4.2.1. Overall

The front-end of this web application will be developed using React JS, a powerful JavaScript library, in combination with three essential technologies: HTML, CSS, and JavaScript. Each of these languages plays a crucial role in creating an interactive, user-friendly, and optimized web experience.

React JS is a JavaScript library developed by Facebook, renowned for its ability to build efficient and flexible user interfaces. React follows a component-based architecture, allowing developers to reuse interface elements, making development faster and more manageable. With React, UI updates are handled intelligently, ensuring only the necessary components are re-rendered, which improves performance and speed, providing a smoother user experience.

HTML (Hypertext Markup Language) forms the fundamental structure of a web page. It defines content elements such as headings, paragraphs, images, and links, creating the backbone of the website's layout.

CSS (Cascading Style Sheets) is responsible for the look and feel of the webpage. Using CSS, developers can control the layout, colors, fonts, and animations, ensuring that the website is visually appealing. CSS also enables responsive design, adapting the layout to different screen sizes and devices, from desktop computers to mobile phones.

JavaScript is a high-level programming language that makes the website dynamic and interactive. With JavaScript, actions such as button clicks, form submissions, or content updates can be processed without reloading the entire page, creating a seamless, fast, and engaging experience for users.

The integration of React JS, HTML, CSS, and JavaScript creates a powerful and flexible front-end ecosystem. This combination not only meets functional requirements but also delivers an optimized and visually attractive user experience, ensuring the application is both fast and interactive.

4.2.2. Application illustration

Due to limited time and resources, the application only builds two diagnostic models for heart disease and stroke.

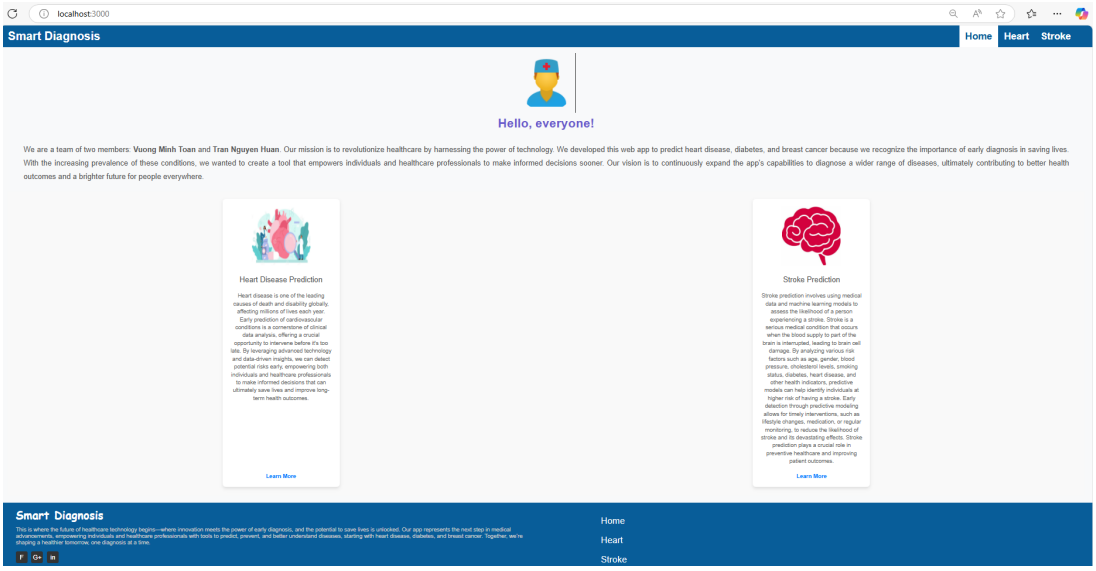

Figure 4.2.1: Homepage Interface

Users can access the homepage of the website, where they will find introductory information about the web application, its purpose, and an overview of the diagnosis process.

From the header section, users can select the health diagnosis they are interested in.

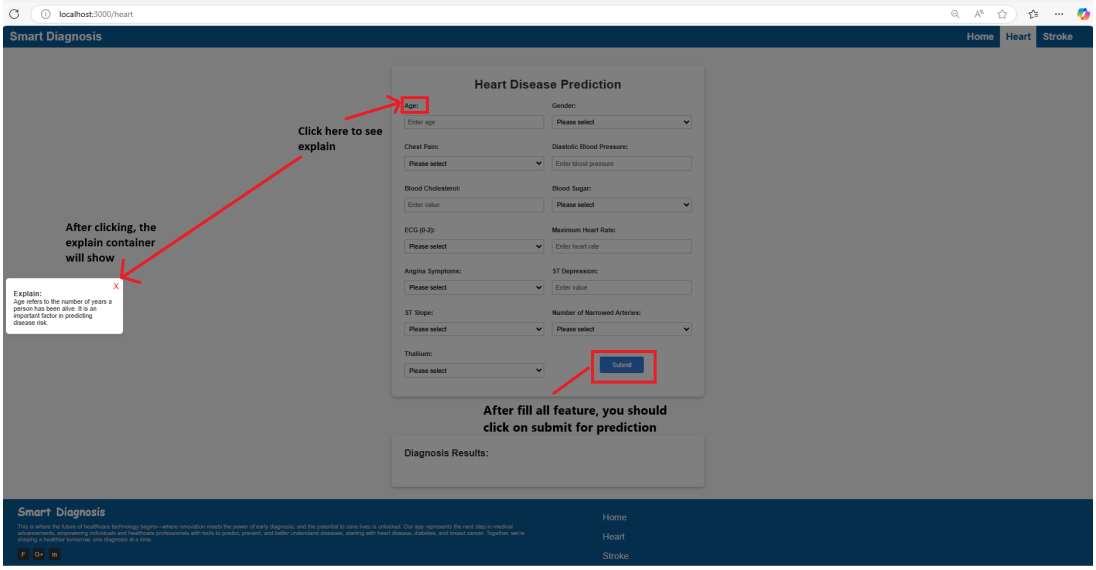

Figure 4.2.2: Heart Diagnosis Interface

They will then input their health information, and if any part is unclear, they can click the title for explanations.

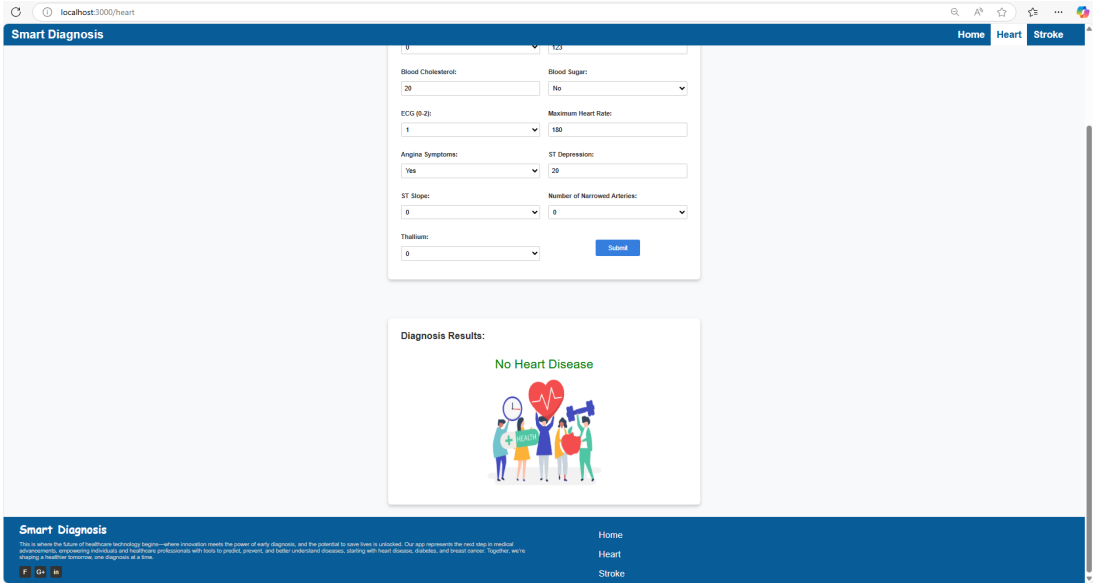

Figure 4.2.3: Heart Diagnosis Prediction Result

Once all the information is entered, users can press the "Submit" button, and the diagnostic results will be displayed in the "Diagnosis Results" section.

4.3. Back end

4.3.1. Overall

After completing the training process, the model is exported and stored as a '.pkl' file. To integrate the trained model into the backend server and seamlessly connect it to the user interface, our team chose Flask, a Python-based framework. The decision to use Flask was driven by its simplicity, flexibility, and ability to scale with the needs of the application.

Flask stands out as a micro-framework that provides just the core features necessary for building web applications, making it lightweight and easy to use. Unlike larger frameworks like Django, Flask allows developers to have greater control over the application structure, ensuring a highly customizable development process. Its minimalistic design enables faster development cycles, which is particularly important for projects that require quick prototyping and iterative adjustments.

Another significant advantage of Flask is its vast ecosystem of extensions. It offers a wide range of built-in and third-party extensions for tasks such as database integration, file uploads, authentication, email handling, and more. These extensions are easy to integrate, making it possible to expand the application's functionality as required without unnecessary complexity.

In summary, Flask's lightweight yet powerful nature, combined with its flexibility and scalability, made it the ideal choice for our project, allowing us to integrate the trained model efficiently and connect it seamlessly to the user interface.

4.3.2. Backend processing workflow

Step 1: Once the backend server successfully establishes a connection with the frontend server, the client sends the input features to the server.

Step 2: The server processes the received feature values and feeds them into the '.pkl' file for diagnostic execution. Since some features may contain data that is not in 'int' or 'float' format, we implemented data conversion for those specific features.

Step 3: The diagnostic results are then sent back from the server to the client.

Section 5

Evaluation results

- Confusion matrix:

```
cm = confusion_matrix(y_test, y_pred_best)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.show()
```

- Accuracy for test:

```
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
accuracy_for_test = accuracy_score(y_test, y_pred)
print('Accuracy for test set with best parameters = {:.2f}'.format(accuracy_for_test))
print('\nClassification Report:\n', classification_report(y_test, y_pred))
confusion_mat = confusion_matrix(y_test, y_pred)
print('\nConfusion Matrix:\n', confusion_mat)
```

- ROC curve:

```
y_pred_prob = best_model.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)
print('AUC = {:.2f}'.format(roc_auc))
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (AUC = {:.2f})'.format(roc_auc))
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
```

```
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate (FPR)')
    plt.ylabel('True Positive Rate (TPR)')
    plt.title('Receiver Operating Characteristic (ROC)')
    plt.legend(loc="lower right")
    plt.show()
```

5.1. Heart Disease Diagnosis

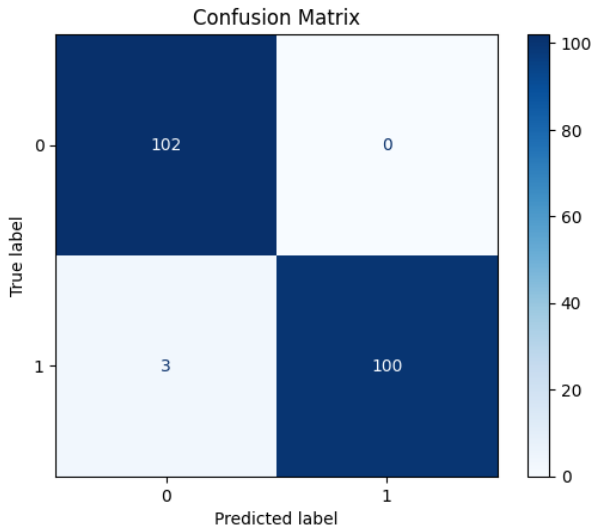

Figure 5.1.1: Confusion Matrix of Heart Disease Model

TP: 102, TN: 100, FP: 3, FN: 0

The model has very high accuracy, nearly 0.99. There are no negative samples misclassified as positive (FP = 0), which is excellent. However, a few positive samples were misclassified as negative (FN = 3). This could be a concern if identifying positive samples is critical.

```
Accuracy for test set with best parameters = 0.99

Classification Report:
              precision    recall  f1-score   support

           0       0.97      1.00      0.99       102
           1       1.00      0.97      0.99       103

    accuracy                           0.99       205
   macro avg       0.99      0.99      0.99       205
weighted avg       0.99      0.99      0.99       205
```

Figure 5.1.2: The Accuracy of Heart Disease Model

The recall for class 1 is slightly lower (0.97), suggesting a small number of positive cases were missed. A accuracy 0.99 indicates that the model is highly reliable for predicting heart disease in the test set. The perfect precision for class 1 ensures no false positives, which is crucial in medical diagnostics. The recall for class 1 (positive cases) could be improved to ensure no missed cases, as false negatives (FN) could be critical in heart disease detection.

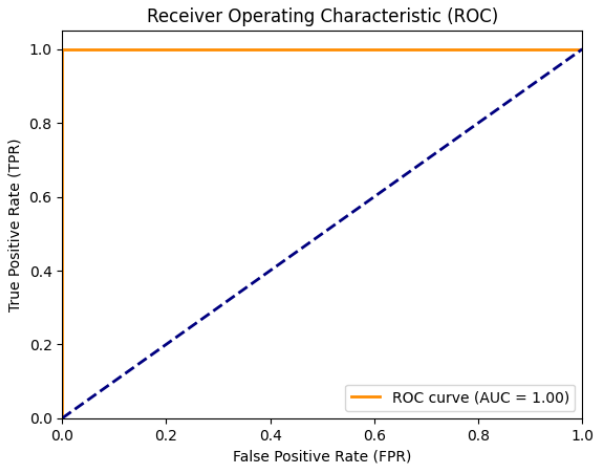

Figure 5.1.3: ROC of Heart Disease Model

The ROC curve shows an AUC of 1.00, indicating perfect discrimination between the positive and negative classes. This means the model is excellent at distinguishing between patients with and without heart disease. The ROC curve reaches the top-left corner, showing that the model achieves a high TPR (sensitivity) while maintaining a low FPR. In this case, FPR is effectively 0, meaning no negative cases are misclassified as positive. A perfect AUC value suggests the model has no trade-offs between sensitivity and specificity. It handles predictions for both classes with maximum accuracy.

The model demonstrates near-perfect accuracy and effectiveness across all metrics, making it an ideal solution for heart disease prediction. Its strengths include zero false positives (FP = 0), which ensures no false alarms, and high performance in predicting the positive class (patients with heart disease), achieving a precision of 1.00 and a recall of 0.97. Additionally, the model boasts a perfect ROC curve (AUC = 1.00), confirming its absolute discrimination ability. While the model only missed a few positive cases (FN = 3), further fine-tuning could improve recall if detecting all cases is critical. Overall, this is an excellent model, highly suitable for implementation in heart disease diagnosis systems, with outstanding reliability and accuracy.

5.2. Stroke Disease Diagnosis

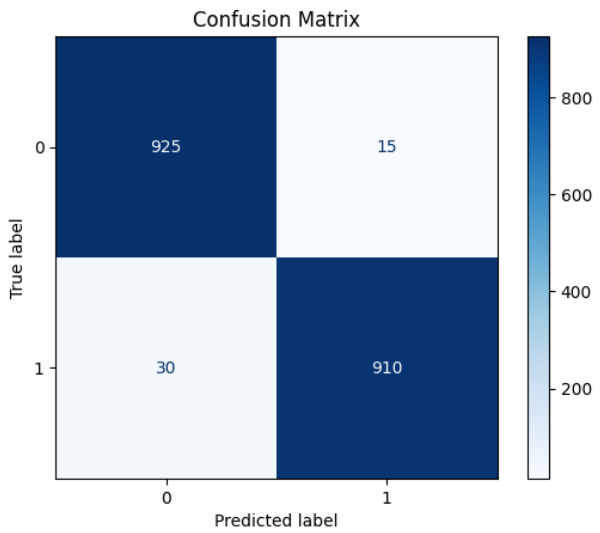

Figure 5.2.1: Confusion Matrix of Stroke Disease Model

TP: 925, TN: 910, FP: 30, FN: 15

The model demonstrates very high accuracy, approximately 0.976. There are very few negative samples misclassified as positive (FP = 12), which indicates strong performance in this regard. However, a small number of positive samples were misclassified as negative (FN = 33). This could be a concern if accurately identifying positive samples is critical.

```
Accuracy for test set with best parameters = 0.97

Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.98      0.97       940
           1       0.98      0.97      0.97       940

    accuracy                           0.97      1880
   macro avg       0.97      0.97      0.97      1880
weighted avg       0.97      0.97      0.97      1880
```

Figure 5.2.2: The Accuracy of Stroke Disease Model

The recall for class 1 is slightly lower (0.96), suggesting a small number of positive cases were missed. An accuracy of 0.98 indicates that the model is highly reliable for predicting outcomes in the test set. The high precision for class 1 (0.99) ensures very few false positives, which is crucial in applications such as medical diagnostics. However, the recall for class 1 (positive cases) could be improved to minimize missed cases, as false negatives (FN) could be critical in scenarios like stroke disease detection or other sensitive domains.

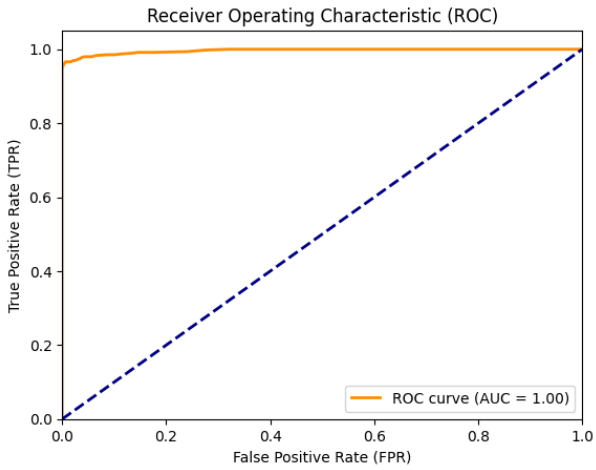

Figure 5.2.3: ROC of Stroke Disease Model

The ROC curve for the stroke disease model demonstrates exceptional performance, with an AUC value of 1.00, indicating perfect classification ability. The curve lies close to the top-left corner, which signifies the model achieves a high true positive rate (TPR) while maintaining a near-zero false positive rate (FPR). This result suggests that the model flawlessly distinguishes between positive and negative cases at all decision thresholds. Such a performance is highly desirable, especially in medical applications like stroke detection, where minimizing errors is critical. The perfect AUC underscores the model's reliability and its potential for deployment in real-world diagnostic scenarios.

The evaluation results from the confusion matrix, classification report, and ROC curve collectively highlight the model's exceptional performance. With an accuracy of 0.98 and an AUC of 1.00, the model demonstrates remarkable reliability in distinguishing between classes. The high precision and recall values, particularly for class 1, indicate that the model is effective in identifying positive cases while minimizing false positives. However, the slightly lower recall for class 1 (0.96) suggests that a small number of positive cases were missed, which could be a concern in sensitive applications like medical diagnostics. Overall, the model's performance is highly robust, making it suitable for critical tasks such as heart disease or stroke detection, with opportunities to further optimize recall to ensure no critical cases are overlooked.